1. Práctica 10: Multitarea Cooperativa

1.1. Objetivos

- Entender la importancia de la multitarea en sistemas empotrados y de tiempo real.
- Desarrollar un modelo cooperativo para la ejecución de diferentes tareas.

1.2. Fundamentos teóricos

Conforme aumenta la complejidad de un sistema, se hace necesaria la ejecución de varias tareas diferentes. Hasta ahora, nuestra solución ha sido tener un gran bucle principal que va ejecutando todo lo necesario. Si son múltiples acciones las que se deben llevar a cabo, se habrá programado una lógica que las vaya haciendo una tras otra en algún orden. Sin embargo, cuando aumenta la complejidad del sistema, esta aproximación acaba resultando inviable. En su lugar, se favorece el desarrolar algún tipo de *framework* que nos permita crear *tareas*, y las ejecute automáticamente cada cierto tiempo.

Especialmente en sistemas empotrados, esta temporalidad es muy importante (y en ocasiones crítica):

- Un generador que debe asegurar una frecuencia exacta de 50Hz.
- Un controlador de vuelo que debe muestrear una unidad de medición inercial a 400Hz.
- Un sistema de airbag que debe comprobar la velocidad y dispararse en menos de 1ms.

Para dar servicio a todos estos sistemas, y cualquier otro, se crean *planificadores*. Los planificadores no son algo que tenga dentro de sí el procesador (como podían ser los periféricos), sino que son responsabilidad del programador, que los programa aprovechando las capacidades del procesador. La principal característica que se explota para ello es la capacidad de temporización del procesador, ya sea a través de los `TIM` `rm0431 18,19,20`, a través del `RTC` `rm0431 25`, o a través (y lo más común) del *ticker* del sistema `SYSTICK` `pm0253 4.4`.

Con estos, se introducen interrupciones periódicas que paran las tareas en curso, y dan paso a que un planificador pueda organizar las sigiuentes tareas a ejecutar.

1.3. Desarrollo de la práctica

Lo primero es entender qué perseguimos, y podemos verlo en la Figura 1. Tener unas tareas (por ejemplo 1, 2 y 3) que se ejecuten, respectivamente, en intervalos diferentes (250, 500 y 1000ms). Para medir el tiempo, utilizaremos el `SYSTICK`, configurado precisamente para que avise cada milisegundo. Por supuesto, si se quiere una granularidad diferente de tiempos, se puede configurar con otros intervalos diferentes.

Como vemos, queda cierto tiempo libre entre la ejecución de tareas. Una cuestión importante de indicar es que, por mucho que consideremos este tipo de sistemas para "tiempo real", tenemos que asegurar siempre que las tareas tardan menos en ejecutarse, del tiempo disponible para la ejecución. De lo contrario, y aunque quizá sea obvio, no dará tiempo a completar todas.

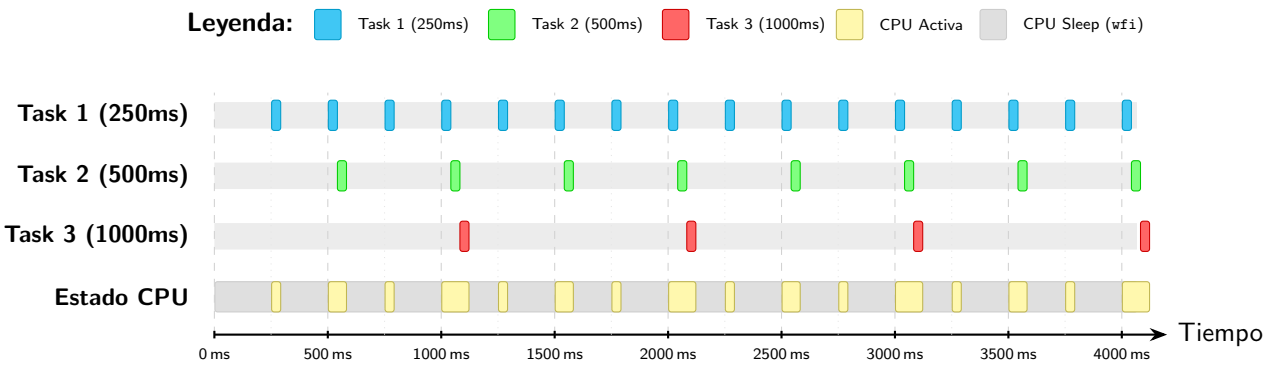


Figura 1: Diagrama de planificación de tareas.

Generalmente, se planifican de esta forma tareas muy críticas, condenando tareas no críticas a poder potencialmente perder "turnos". No es el objetivo en cualquier caso de esta práctica, ya que nuestras tareas serán muy sencillas, pero hay que tenerlo en cuenta.

Nuestras tareas simplemente parpadearán los 3 LEDs disponibles en la placa (El LED rojo en `PA7`, el azul el `PA5`, y el verde en `PB1`, recuerda que deberás inicializar los `GPIO` correspondientes).

Nuestro código será extremadamente simple (al menos el principal, el resto quizá sea debatible). La estructura es la siguiente:

```c
void SysTick_UserCallback() {
    scheduler();
}

int main(void) {
    SysTick_Init();
    GPIO_Init();
    scheduler_init();

    create_task(Task_BlinkBlue, 250);
    create_task(Task_BlinkGreen, 500);
    create_task(Task_BlinkRed, 1000);

    // Scheduler
    while (1) {
        // Low-power sleep: Wait for the next SysTick interrupt before executing the scheduler loop again
        __asm volatile ("wfi");
        dispacher();
    }

    return 0;
}
```

Disponemos de una función `SysTick_UserCallback` que se llamará automáticamente en cada *tick* del `SYSTICK`, y organizará las tareas a ejecutar. Por otro lado, una función `main` que creará las tareas, y ejecutará el *despachador* para irlas ejecutando según estén listas. El diagrama general de ejecución se puede ver en la Figura 2.

Es decir, la función del *scheduler* es la de organizar qué tareas se pueden ejecutar, si es que ya les toca su "hora". Por su parte, el *dispatcher* se encargará de lanzarlas a ejecución. La lógica que pongamos dentro de cada uno será lo que determine qué tareas tienen más o menos

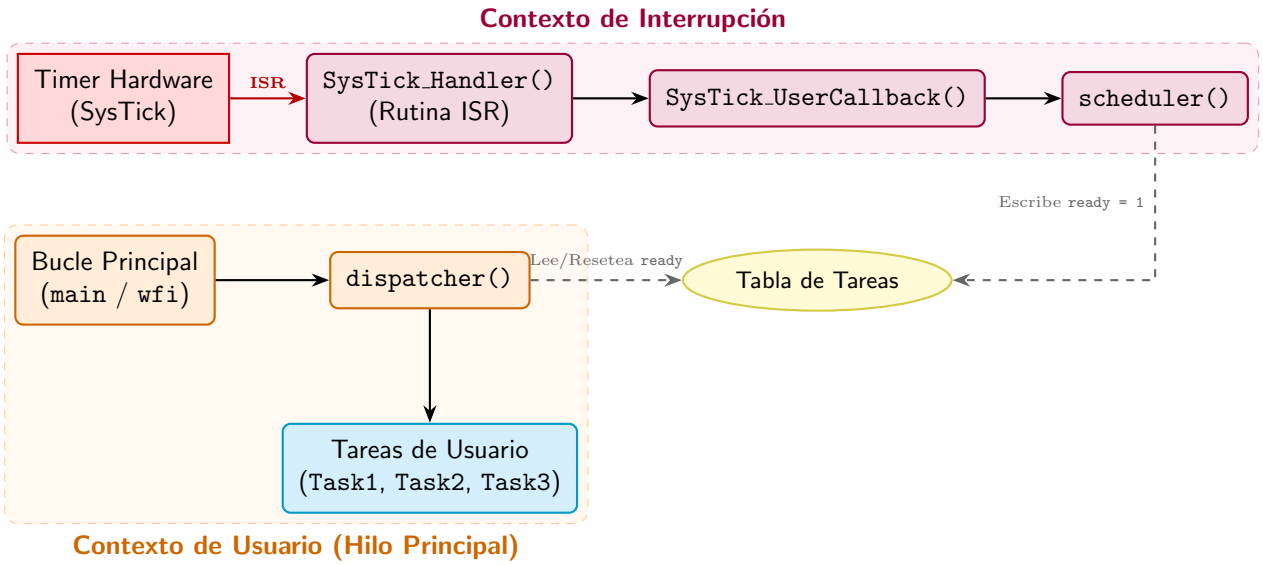


Figura 2: Arquitectura de llamadas a funciones e interacción de datos entre ISR e Hilo Principal.

prioridad. El *ticker* de sistema será el que vaya dictando cuándo se ejecutan estas dos funciones, ya que llamará al *scheduler* en cada *tick*, y a la vez despertará al procesador en caso de estar esperando en la instrucción `__asm volatile ("wfi");` (*wait for interrupt*).

Lo primero que necesitamos es modificar ligeramente el fichero `bsp.h` (separándolo en `bsp.h` y `bsp.c`) para incluir la llamada a la función `SysTick_UserCallback`. Así tendremos el aviso de que ha habido un *tick*. (No se puede hacer todo en `bsp.h` ya que saldrán errores de redefinición de funciones y/o variables)

```c
//en bsp.h, quitamos la implementacion del Handler, y
//declaramos la variable msTicks como extern volatile
//para indicar que se define en el bsp.c
extern volatile uint32_t msTicks;

void SysTick_UserCallback(void);
void SysTick_Handler(void);


//código para bsp.c
#include "bsp.h"

volatile uint32_t msTicks = 0;

__attribute__((weak)) void SysTick_UserCallback(void) {
    // Default empty implementation
}

void SysTick_Handler(void) {
    msTicks++;
    SysTick_UserCallback();
}
```

Importante notar dos cosas: La variable msTicks que se define como externa en el `bsp.h` para que otros archivos que lo importen (como `main.c`) no lo redefinan, creando así conflictos en compilación. Por otro lado, la definición de `SysTick_UserCallback` como `__attribute__((weak))`, lo cual permite:

1. Tener ya la llamada hecha desde `bsp.c`.
2. Poder redefinir la función desde otro archivo (en este caso `main.c`) para que se llame a esa en su lugar.

Una vez modificado ya podemos atacar a `main.c`. Lo primero es definir las estructuras de tarea. Una tarea consta de una función a la que llamar, y datos sobre cada cuánto se tiene que llamar, y hace cuánto se ha llamado por última vez, para que el planificador pueda gestionar su ejecución:

```
#define MAX_TASKS 10

// Task structure defining what a task is
typedef struct {
    void (*task_function)(void); // Pointer to the task function
    uint32_t period_ms;          // How often the task should execute (in ms)
    uint32_t last_run;           // The sys_ticks timestamp when it last
        executed
    uint8_t ready;               // True if the task should run
} Task_TypeDef;

// Tasks in our program
static Task_TypeDef tasks[MAX_TASKS] = {0};
```

Las tareas en este caso son bien sencillas, simplemente parpadean unos LEDs:

```
void Task_BlinkGreen(void) {
    GPIO_PIN_TOGGLE(GPIOB, 1);
}

void Task_BlinkBlue(void) {
    GPIO_PIN_TOGGLE(GPIOA, 5);
}

void Task_BlinkRed(void) {
    GPIO_PIN_TOGGLE(GPIOA, 7);
}
```

Ahora toca crear las funciones para gestionar tareas. Observa que `create_task` recibe una función como argumento, que se guarda internamente en su estructura

```
uint32_t create_task( void (*pfunction)(void), uint32_t period ) {
    uint32_t id;

    // ... completar buscar una entrada libre en la tabla de tareas
    id =

    // ... completar inicializar los campos de la tarea

    return id;
}

void delete_task (uint32_t id ) {
    // ... completar inicializar la estructura a 0
    // ... para luego poder identificar si hay o no tareas
    // ... o si están o no listas
}

void scheduler_init (void) {
    // ... ejecuta delete_task para todas las tareas, a fin
```

```
    // ... de reiniciarlas
}
```

Con las tareas creadas, ya sólo nos queda crear el planificador y el despachador. Para el planificador, basta recorrer la lista de tareas y, si ya le toca ejecutarse (comprueba que desde la última vez que se ejecutó, ya ha pasado el tiempo suficiente), la marcamos como *ready*:

```
void scheduler (void) {
    // ... completar mirar el momento de tiempo actual (GetTick())
    // ... recorrer la lista de tareas,
    // ... si la trea existe, ver si le toca ya (en función
    // ... del tiempo) ejecutarse, y en ese caso marcar como
    // ... ready y ajustar valores para que no se dispare de nuevo
    // ... hasta que le vuelva a tocar
}
```

El dispatcher, simplemente lo que hará será ejecutar las tareas que estén listas:

```
void dispacher (void) {
    // ... completar recorrer la lista de tareas
    // ... completar si la tarea está lista, ejecutar su función
    // y marcar como no lista
}
```

Y listo! Ya solo nos falta conectar el *callback* del `SYSTICK` para que llame a nuestro *scheduler* cada vez que se dispare:

```
void SysTick_UserCallback() {
    scheduler();
}
```

Si hemos hecho bien todo y lo ejecutamos, deberíamos ver parpadear los 3 leds en diferentes frecuencias. Por detrás, lo que está ocurriendo es que 1000 veces por segundo se está disparando el *scheduler*, y a continuación el *dispatcher* al despertar el procesador de su espera. En caso de encontrar funciones para ejecutar, las lanza. Si no, se vuelve a dormir. Si una función tarda más de un hueco (o *timeslot*) en ejecutarse, simplemente el *dispatcher* no se lanzará en ese ciclo. Podemos ver este detalle en la Figura 3.

1.3.1. Fragilidad de la aproximación

Añade una tarea que no funcione "bien" (por ejemplo que entre en un bucle de espera con `Delay_ms`). ¿Qué pasa con la ejecución del resto? Comprueba también que pasa si esta tarea está la primera o la última. ¿Cómo piensas que podría solucionarse este problema?

1.4. Para hacer más

En sistemas complejos, y para evitar bloqueos, se suelen introducir prioridades a las tareas, a fin de que las tareas más prioritarias (del sistema) puedan pasar primero. Incluye un sistema de prioridades a las tareas, donde tareas con una prioridad más alta se ejecuten siempre primero. (**NOTA:** al ser un planificador *cooperativo*, podrían bloquearse igualmente si una tarea no cede su turno, no te preocupes si ocurre).

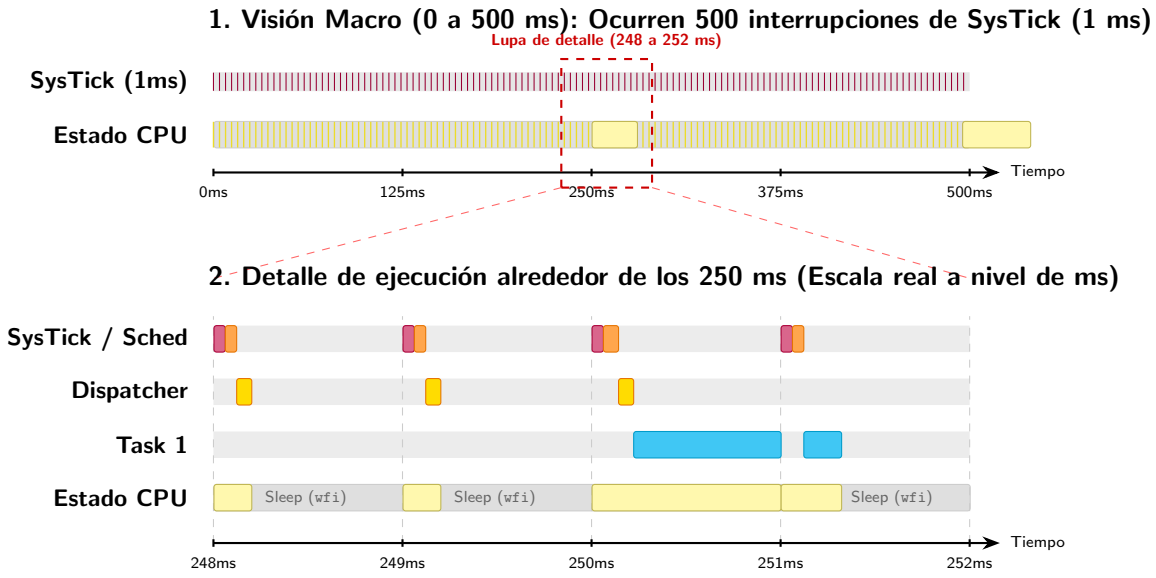


Figura 3: Detalle de ejecución de los diferentes elementos de nuestro planificador, con una tarea que tarda más de un *slot*